53. Maximum subarray: (Kadane's Algorithm)

Given an integer array `nums`, find the subarray with the largest sum, and return *its sum*.

Example 1:

```
Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
Explanation: The subarray [4,-1,2,1] has the largest sum 6.
```

Example 2:

```
Input: nums = [1]
Output: 1
Explanation: The subarray [1] has the largest sum 1.
```

Example 3:

```
Input: nums = [5,4,-1,7,8]
Output: 23
Explanation: The subarray [5,4,-1,7,8] has the largest sum
23.
```

Constraints:

- `1 <= nums.length <= 105`
- `104 <= nums[i] <= 104`

Follow-up: If you have figured out the `O(n)` solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

Solution: Time = O(n) and O(1)

```python
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
```

```
        curSum = 0
        maxSum = min(nums)
        for i in nums:
            curSum += i
            maxSum = max(curSum, maxSum)
            if(curSum < 0):
                curSum = 0
        return maxSum
```